Laporan Repositori: 3D Mapping dengan Quadcopter (CoppeliaSim + Python)

Author(s): Oktavian Rifki Danendra (NRP 5022231107)
Teacher: Muhammad Qomaruz Zaman, S.T., M.T., Ph.D
Class name: Sistem Robotika (SRO) – Evaluasi Tengah Semester
Video link: `https://drive.google.com/drive/folders/1Ik7VPm2pZWIu-oK68S5vS6aB16m1mVCD`
GitHub link: `https://github.com/rfkdndraa/Autonomous-Robotics-Systems-Quadcopter-Mapping`

Pendahuluan

Laporan ini membahas seluruh file yang ada di repositori `Autonomous-Robotics-Systems-Quadcopter-Mapping` beserta penjelasan singkat fungsi masing-masing. Repositori tersebut berisi dokumentasi ETS mata kuliah Sistem Robotika: sebuah quadcopter di simulator CoppeliaSim [**?**] terbang otomatis mengelilingi arena berisi kotak, silinder, dan kerucut sambil memindai lingkungan dengan sensor `fast3DLaserScanner`. Data kedalaman hasil pindaian diubah menjadi *point cloud* 3D, direduksi menjadi voxel [**?**], lalu divisualisasikan dengan pewarnaan berdasarkan ketinggian. Pemrosesan dilakukan di Python (NumPy [**?**] dan Matplotlib [**?**]) yang terhubung ke CoppeliaSim melalui ZMQ Remote API.

Struktur Repositori

Repositori berisi sembilan file. Ringkasannya ada pada Tabel 1, dan pembahasan tiap kelompok file diuraikan pada bagian berikutnya.

Table 1: Daftar file di repositori dan fungsinya

File	Jenis	Fungsi singkat
`5022231107_Quadcopter_Mapping.py`	Skrip Python	Program utama: terbang otomatis, pemindaian, pembuatan voxel, dan plot peta.
`5022231107_Quadcopter_Scene.ttt`	Scene CoppeliaSim	Dunia simulasi: quadcopter, dummy `target`, scanner, dan objek penghalang.
`Quadcopter_Lua_Default.txt`	Skrip Lua (bawaan)	Pengendali terbang quadcopter dan efek propeller.
`Fast3DLaserScanner_Lua_Default.txt`	Skrip Lua (bawaan)	Pembacaan sensor *fast 3D laser scanner*.
`README.md`	Dokumentasi	Penjelasan fitur, cara pakai, konfigurasi, dan pemecahan masalah.
`hasil_2d_map.png`	Gambar hasil	Peta 2D tampak atas, pola *lawnmower*.
`hasil_3d_map.png`	Gambar hasil	Peta 3D voxel, pola *lawnmower*.
`hasil_2d_map_figure8.png`	Gambar hasil	Peta 2D tampak atas, pola angka 8.
`hasil_3d_map_figure8.png`	Gambar hasil	Peta 3D voxel, pola angka 8.

Pembahasan File

1. Skrip utama: `5022231107_Quadcopter_Mapping.py`

Ini adalah file inti (sekitar 378 baris). Skrip terhubung ke CoppeliaSim lewat `RemoteAPIClient`, menjalankan simulasi dalam mode *stepping*, lalu menerbangkan drone sambil mengambil data sensor. Setelah simulasi dihentikan, data diproses dan disimpan sebagai `pointcloud_raw.npy`, `pointcloud_voxel.npy`, serta gambar peta. Fungsi-fungsi utamanya dirangkum pada Tabel 2.

Table 2: Fungsi utama pada skrip Python

Fungsi	Peran
`figure8_waypoints`, `lawnmower_waypoints`, `make_waypoints`	Membuat daftar waypoint sesuai `PATH_MODE`: angka 8 atau *lawnmower*.
`find_scanner`, `get_sensors`	Mencari model `fast3DLaserScanner`, memasangnya sebagai anak `Quadcopter` bila belum, lalu mengambil parameter *near*, *far*, dan FOV tiap vision sensor.
`find_obstacles`, `avoidance`	Mendaftar semua shape penghalang (kecuali drone dan lantai) dan menghitung dorongan menghindar.
`goto, fly_and_scan`	Menggerakkan dummy `target` menuju waypoint sambil menghindar, dan mengambil frame depth setiap `SCAN_EVERY` langkah.
`capture, frame_to_world`	Mengubah satu frame depth menjadi titik 3D, lalu memindahkannya ke koordinat dunia.
`choose_convention`	Memilih konvensi sumbu kamera (`flip_x` atau `flip_y`) secara otomatis.
`voxel_downsample`, `remove_isolated`	*Post-processing*: satu titik per voxel dan pembuangan voxel terpencil (noise).
`occupancy_2d, plot_all`	Membuat peta 2D tampak atas, plot 3D berwarna menurut tinggi, dan menyimpan gambar.

Pola terbang. Dua pola tersedia. Pola angka 8 memakai lemniscate Gerono dengan ketinggian tetap h (`ALTITUDE`) dan N putaran:

$$x(t) = a \sin t, \qquad y(t) = b \sin 2t, \qquad z(t) = h, \qquad t \in [0,\, 2\pi N] \tag{1}$$

dengan $a =$ `FIG8_A` dan $b =$ `FIG8_B`. Pola *lawnmower* menyapu area dengan lajur sejajar berjarak `LANE_SPACING` dan arah bolak-balik, sebuah pola cakupan area yang umum dipakai pada robotika [**?**].

Penghindaran halangan. Fungsi `avoidance` memakai pendekatan *potential field* [**?**]. Untuk setiap penghalang yang lebih dekat dari d_{safe} (`SAFE_DIST`), jarak terdekat dihitung dengan `sim.checkDistance`. Bobot dan dorongan totalnya:

$$w = \frac{d_{\text{safe}} - d}{d_{\text{safe}}}, \qquad \mathbf{u} = G\,\Delta s \sum_k w_k \left(\hat{\mathbf{a}}_k + \kappa\, \hat{\mathbf{t}}_k\right) \tag{2}$$

dengan $\hat{\mathbf{a}}_k$ vektor satuan menjauhi penghalang, $\hat{\mathbf{t}}_k$ komponen tangen yang searah tujuan (agar drone menyusuri sisi penghalang), $G =$ `AVOID_GAIN`, $\kappa =$ `SLIDE_GAIN`, dan $\Delta s =$ `STEP_DIST`. Besarnya dorongan dibatasi oleh `MAX_PUSH`.

Dari depth ke point cloud. Tiap piksel depth (u, v) dengan kedalaman z diubah menjadi titik di koordinat sensor menggunakan model kamera *pinhole*, dengan resolusi $w \times h$ dan sudut pandang θ:

$$f = \frac{\max(w,h)/2}{\tan(\theta/2)}, \qquad \mathbf{p}_s = \left[\frac{u - (w-1)/2}{f}\, z,\;\; \frac{v - (h-1)/2}{f}\, z,\;\; z\right]^{\top} \tag{3}$$

Titik yang lebih dekat dari `MIN_RANGE` (badan drone) atau lebih jauh dari `MAX_RANGE` dibuang. Titik kemudian dipindahkan ke koordinat dunia memakai pose sensor $[\mathbf{R}\,|\,\mathbf{t}]$ saat frame diambil:

$$\mathbf{p}_w = \mathbf{R}\,\mathrm{diag}(s_x, s_y, 1)\,\mathbf{p}_s + \mathbf{t} \tag{4}$$

Faktor (s_x, s_y) adalah konvensi sumbu kamera. Karena konvensi ini tidak pasti, `choose_convention` mencoba `flip_x` dan `flip_y`, lalu memilih yang menghasilkan jumlah voxel objek paling sedikit, yaitu yang membuat objek dari banyak sudut pandang menumpuk di tempat yang sama.

Voxel dan pembersihan noise. Titik dikelompokkan ke voxel berukuran Δ (`VOXEL_SIZE` = 0,10 m), dan tiap voxel diwakili satu titik di pusatnya:

$$\mathbf{i} = \lfloor \mathbf{p}_w / \Delta \rfloor, \qquad \mathbf{c} = \left(\mathbf{i} + \tfrac{1}{2}\right)\Delta \tag{5}$$

Fungsi `remove_isolated` lalu membuang voxel yang memiliki kurang dari `MIN_NEIGHBORS` tetangga di antara 26 voxel sekelilingnya, sehingga titik melayang akibat noise hilang sementara permukaan lantai dan badan objek tetap utuh.

Pewarnaan dan peta 2D. Voxel dengan $z \leq$ `GROUND_Z` (0,15 m) dianggap tanah dan diberi satu warna tetap, sedangkan voxel di atasnya dianggap objek dan diwarnai menurut tinggi dengan colormap `plasma` lengkap dengan *colorbar*. Peta 2D diperoleh dengan memproyeksikan semua voxel di atas lantai ke bidang XY (`occupancy_2d`), lalu jalur drone digambar sebagai garis biru.

2. Scene CoppeliaSim: `5022231107_Quadcopter_Scene.ttt`

File ini adalah dunia simulasi yang dibuka di CoppeliaSim (diuji di versi Edu 4.10). Skrip Python mengharapkan objek berikut ada di scene (Tabel 3). Aturan paling penting: `fast3DLaserScanner` harus menjadi anak `Quadcopter`, diletakkan sedikit di bawah badan drone, dan menghadap ke bawah. Skrip memiliki *preflight check* yang menolak berjalan bila scanner menumpuk di badan drone.

Table 3: Objek yang dibutuhkan skrip di dalam scene

`Objek`	**Keterangan**
`/Quadcopter`	Model quadcopter bawaan CoppeliaSim.
`/target`	Dummy yang diikuti drone; skrip menggerakkan dummy ini.
`/Quadcopter/fast3DLaserScanner`	Sensor pemindai, harus menjadi anak Quadcopter.
`Cuboid, Cylinder, Cone, dinding`	Penghalang yang dipetakan.

3. Skrip Lua bawaan

Dua file `.txt` berikut, dilihat dari nama dan isinya, adalah skrip Lua bawaan CoppeliaSim untuk model quadcopter dan *fast 3D laser scanner* di scene. Keduanya menjadi dasar kerja sistem.

`Quadcopter_Lua_Default.txt.` Skrip ini adalah pengendali terbang. Pada `sysCall_init` ia melepas dummy `target` dari induknya dan membuat objek partikel untuk keempat propeller. Pada `sysCall_actuation` dihitung tiga kontrol:

1. **Kontrol vertikal** berbentuk PID pada galat tinggi terhadap target, ditambah umpan balik kecepatan vertikal:
$$T = 5{,}45 + K_p e + K_i \sum e + K_d(e - e_{\text{prev}}) + K_v \dot{z}, \qquad e = z_{\text{target}} - z \tag{6}$$
dengan $K_p = 2$, $K_i = 0$, $K_d = 0$, dan $K_v = -2$ pada skrip.
2. **Kontrol horizontal**: koreksi sudut roll (α) dan pitch (β) dihitung dari kemiringan badan dan posisi target relatif terhadap drone.
3. **Kontrol rotasi**: koreksi yaw dari selisih orientasi drone terhadap target.

Keempat koreksi digabung menjadi kecepatan tiap propeller melalui `handlePropeller`, misalnya $T(1 - \alpha_c + \beta_c + \psi_c)$ untuk propeller pertama. Fungsi tersebut juga menghasilkan partikel (visual dan gaya reaktif) serta memutar sendi propeller. Komentar di dalam skrip menyebut algoritma ini sebagai contoh sederhana yang belum optimal; pada proyek ini ia dipakai apa adanya untuk mengikuti dummy `target`.

`Fast3DLaserScanner_Lua_Default.txt.` Skrip ini mengatur dan membaca sensor. Pada `sysCall_init`, jangkauan maksimum diset 5 m dan sudut pindai 45°, lalu dibuat tampilan melayang (*floating view*) dan objek gambar titik merah. Pada `sysCall_sensing`, data kedalaman dibaca dengan `sim.readVisionSensor`, tiap titik yang lebih dekat dari jangkauan maksimum dimasukkan ke `measuredData` (relatif terhadap basis sensor), dan titik-titik itu digambar di dunia simulasi sebagai titik merah. Skrip Python tidak memakai `measuredData`; ia membaca depth langsung lewat `sim.getVisionSensorDepth` dan melakukan transformasinya sendiri.

4. Dokumentasi: `README.md`

README menjelaskan tujuan proyek dalam bahasa Indonesia, pemetaan ketentuan tugas ke implementasi, kebutuhan perangkat lunak, struktur scene, cara menjalankan, tabel parameter konfigurasi, dan tabel pemecahan masalah. Pemetaan tingkat tugas pada README dirangkum pada Tabel 4.

Parameter yang paling sering diubah dirangkum pada Tabel 5. Nilai di tabel diambil dari bagian konfigurasi skrip Python.

Table 4: Ketentuan tugas dan implementasinya (disarikan dari README)

Tingkat	Ketentuan tugas	Implementasi
C	Mengontrol drone dan menampilkan jalur	Drone mengikuti dummy `target` sepanjang waypoint; jalur digambar sebagai garis biru.
BC	Scan 2D	Peta 2D tampak atas (`occupancy_2d`).
B	Scan 3D	Frame depth diubah menjadi point cloud 3D di koordinat dunia.
AB	Reduksi titik menjadi voxel	`voxel_downsample` dan `remove_isolated`.
A	Color mapping ketinggian	Tanah satu warna, objek diwarnai menurut tinggi (`plasma`) dengan colorbar.

Table 5: Parameter konfigurasi utama pada skrip (nilai default di kode)

`Parameter`	**Nilai**	**Fungsi**
`PATH_MODE`	"lawnmower"	Pilihan pola terbang: `"figure8"` atau `"lawnmower"`.
`FIG8_LAPS`	3	Jumlah putaran angka 8.
`ALTITUDE`	0,8 m	Ketinggian terbang.
`LANE_SPACING`	0,30 m	Jarak antar lajur mode *lawnmower*.
`SAFE_DIST`	0,9 m	Jarak mulai menghindar dari penghalang.
`SCAN_EVERY`	6	Ambil data depth tiap N langkah simulasi.
`VOXEL_SIZE`	0,10 m	Ukuran voxel.
`MIN_RANGE / MAX_RANGE`	0,3 / 6,0 m	Batas jarak titik yang dipakai.
`MIN_NEIGHBORS`	2	Voxel dengan tetangga lebih sedikit dari ini dibuang.
`GROUND_Z`	0,15 m	Batas tinggi voxel tanah dan objek.

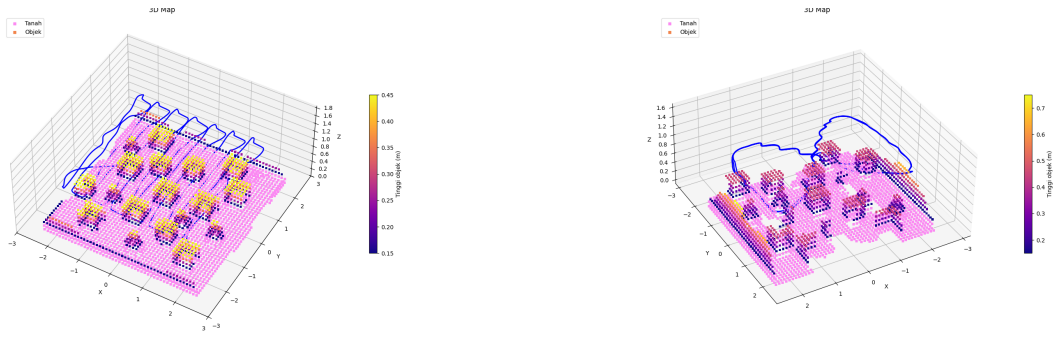
5. Gambar hasil

Empat file PNG adalah keluaran skrip. Skrip selalu menyimpan ke nama `hasil_2d_map.png` dan `hasil_3d_map.png`; versi dengan akhiran `_figure8` adalah hasil pola angka 8 dan namanya tidak dihasilkan langsung oleh skrip versi ini. Hasil dibahas pada bagian berikut.

Hasil dan Pembahasan

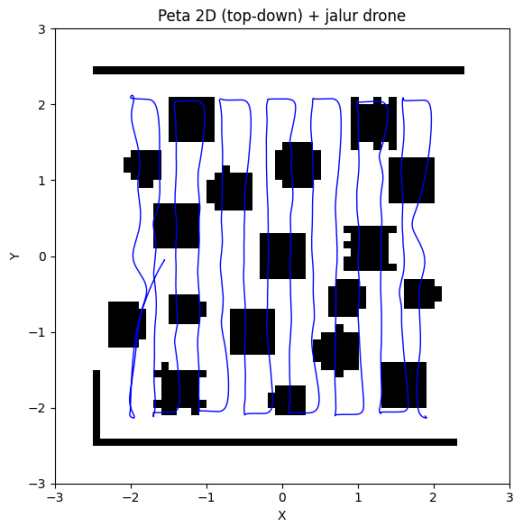
Gambar 1 dan 2 membandingkan hasil kedua pola terbang pada scene yang sama.

Pada pola *lawnmower*, jalur drone berupa lajur-lajur sejajar yang menutupi seluruh arena, sehingga lantai dan hampir semua objek terpetakan secara rapat (Gambar 1a dan 2a). Pada pola angka 8, jalur hanya melewati bagian tengah dan dua lobus di kiri dan kanan, sehingga cakupan lantai lebih berlubang dan beberapa objek hanya terekam sebagian (Gambar 1b dan 2b). Hal ini sesuai dengan sifat kedua pola: *lawnmower* dirancang untuk cakupan penuh, sedangkan angka 8 lebih cocok untuk menunjukkan kendali lintasan yang mulus. Jalur biru tampak bergelombang di dekat objek, yaitu penyimpangan dari garis lajur yang konsisten dengan mekanisme penghindaran halangan; setelah itu drone kembali ke lajur.

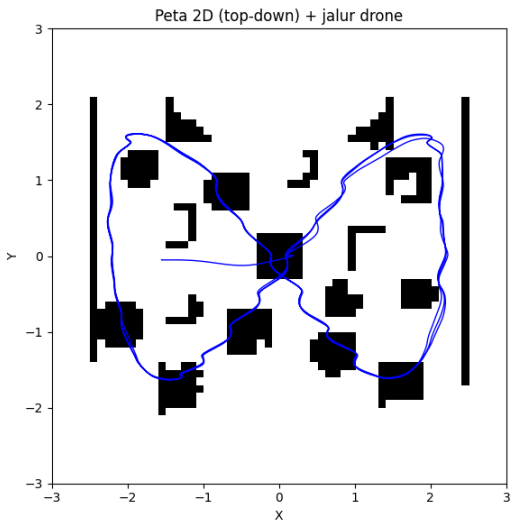


(a) Pola *lawnmower* (b) Pola angka 8

Figure 1: Peta 3D voxel. Tanah berwarna merah muda, objek berwarna menurut tinggi, dan garis biru adalah jalur drone.



(a) Pola *lawnmower*



(b) Pola angka 8

Figure 2: Peta 2D tampak atas (area hitam adalah proyeksi objek di atas lantai) dengan jalur drone berwarna biru.

Alur Kerja Sistem

1. Skrip membuat daftar waypoint (angka 8 atau *lawnmower*) dan menemukan scanner serta daftar penghalang di scene.
2. Dummy `target` digerakkan menuju waypoint; pengendali Lua menerbangkan drone mengikuti `target`, sementara `avoidance` menambahkan dorongan menjauhi penghalang.
3. Setiap `SCAN_EVERY` langkah, satu frame depth diambil dan diubah menjadi titik 3D (Persamaan 3) lalu ke koordinat dunia (Persamaan 4).
4. Setelah terbang, simulasi dihentikan dan konvensi sumbu kamera dipilih otomatis.
5. Semua titik direduksi menjadi voxel (Persamaan 5), voxel terpencil dibuang, lalu peta 3D dan 2D diplot dan disimpan.

Kesimpulan

Repositori ini memuat satu sistem pemetaan 3D yang lengkap: scene simulasi, dua skrip Lua bawaan yang menggerakkan drone dan membaca sensor, satu skrip Python yang mengorkestrasi penerbangan otomatis dan seluruh pemrosesan data, serta dokumentasi dan gambar hasil. Menurut README, kelima tingkat ketentuan tugas (C sampai A) dipenuhi melalui jalur drone, peta 2D, point cloud 3D, voxel dengan pembersihan noise, dan pewarnaan menurut ketinggian. Perbandingan dua pola terbang menunjukkan bahwa pola *lawnmower* memberi cakupan arena yang jauh lebih lengkap daripada pola angka 8.